Past Exams & Lab Exercises

- Top-0.55 nucleus sampling: if we have 0.5 for "the", 0.4 for "a", 0.1 for "b", we will have $0.5/(0.5+0.4)$ probability of generating "the", $0.4/(0.5+0.4)$ probability of extracting "a". We don't slice "a"'s probability, we are allowed to go over $p = 0.55$
- In unigram model highest perplexity given by sequences with lowest average probability (e.g. if no <UNK> token in the dataset then sequences w/ most OOV tokens)
- Beam Search switches between branches of generation tree depending on how sequence prob. changes, increasing beam size can find higher prob. sequences that we couldn't find with lower beam sizes (greedy heuristic, model vs generation (1) probability)
- Add <start> and <stop> tokens to start and end of each sequence in dataset
- Add <pad> here and there in RNNs input to train for different length sequences
- Increasing vocabulary size increases perplexity, which goes down as models improve
- BERT: base (110M, 12 layers w/ 12 heads) and large (340M, 24 layers w/ 16 head)
- In DistilBert compare w/ cosine distance the last hidden layer sentence embeddings of BERT and distillation to align further
- SBERT: sequence -> BERT + pooling (token embeddings -> single sentence embedding) -> cosine similarity with other single sentence embedding
- Reward accuracy > 0.5 indicates the model is successfully learning from preferences
- Keep prompt templates as unbiased as possible (especially for In-context learning)
- KV cache so at each generation w/ attention don't re-compute K, V, $O(n)$ vs $O(n^2)$
- ICL and CoT address diff. bottlenecks, 1st task specification, 2nd reasoning

Neural Classifiers

- **Word representations**: sparse (OHE, no contextual/positional awareness) or dense (embeddings in smaller space than vocab, similar meaning words similar vectors). Word2vec dense word vectors don't generalize better to unseen words than sparse
- **Sum/avg/max-pool models**: aggregate embeddings of input words and multiply with weight matrix, map result to classes (sigmoid for single class, softmax for multi-class (text generation)). Word embeddings and weights learned w/ backpropagation
- **Context representations**: rely on context in which words occur to learn meaning
- **Word2vec CBOW**: predict missing word from its context, sum embeddings of words in the context, project it on the vocab space using a matrix, softmax maps each vocab word to a probability. Model is trained to minimize NLL of the missing word. Not good at capturing rare word relationships, quick to train. Context length hyperparam
- **Word2vec Skip-gram**: predict vector representing the context around a word, objective $max P(x_{t-1}, x_{t+1} | x_t) = max(log P(x_{t-1}|x_t) + log P(x_{t+1}|x_t))$. Captures nuances, dynamic context length changing at each iteration to learn all dependencies, max window length hyperparameter. FastText improves it with char n-gram
- **GloVe**: $X \in \mathbb{R}^{V \times V}$ global word-context co-occurence matrix (X_{ij} how many times word i in context of word j in the whole dataset), objective $min \sum_{i,j} f(X_{ij})((w_i^T w_j + b_i + b_j) - log X_{ij})^2$, biases and $f(X_{ij})$ account for very common words and pairs, w_i, w_j learned word embeddings. Recognizes synonyms better than word2vec because it uses global statistics

Language Models

- **Language Model**: probabilistic model of a sequence of tokens $P(w_1, \dots, w_n) = \prod_i P(w_i | w_1, \dots, w_{i-1})$
- **Count-based (n-gram) Model**: estimate probabilities by counting occurrences of sequences, $P(x_2|x_1, x_0) = \frac{C(x_2, x_1, x_0)}{C(x_1, x_0)}$, k-th order markov assumption makes it feasible to model long sequences (considers only the last k words in the sequence, $P(w_1, \dots, w_n) \approx \prod_i P(w_i | w_{i-1}, \dots, w_{i-k})$), memory grows combinatorially
- **Unigram model**: $p(x_1 x_2 \dots x_n) = \Pi_{i=1}^n p(x_i) = \Pi_{i=1}^n C(x_i)/\#\ tokens\ in\ dataset$
- **Extrinsic evaluation**: on downstream task not part of training, hard to optimize
- **Entropy**: $H(W|M) = \frac{1}{|W|} \sum_{w \in W} -log_2 P(w|M)$, W input, M model
- **Perplexity**: how well LM predicts sample sentences $S = \{S^1, \dots, S^m\}$, $ppl(S) = 2^x$, $x = -\frac{1}{m} \sum_{i=1}^m log_2 P(S^i)$, $P(S^i) = \prod_j P(w_j | w_1, \dots, w_{j-1})$, exponentiated token-level NLL with cross-entropy. If we assign equal probability to all words $ppl(S) = |V|$, measures a model's uncertainty about the next word (k perplexity = model being confused among k tokens on average), easy standardized metric, requires train/test set domain match, cheated by predicting common tokens
- **Single Sequence Perplexity**: $ppl(x_1 x_2 \dots x_n) = p(x_1 x_2 \dots x_n)^{-1/n}$, mean NLL
- **Exponential Decay of Counts**: sequence counts follow power law, very long ones have few examples to learn from, some sequences will never be predicted (count=0) and have infinite perplexity (not interpretable), keep low N-gram size
- **Laplace Smoothing**: $+\alpha$ to all counts, normalize, $P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha |V|}$
- **Discounting Smoothing**: redistribute probability mass from observed n-grams to unobserved ones by removing an absolute amount of counts, $\alpha(w_{i-1})$ is the leftover mass, $P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) - d}{C(w_{i-1})}$ if $C(w_{i-1}, w_i) > 0$ else $\alpha(w_{i-1}) \frac{P(w_i)}{\sum_{w'} P(w')} \forall w'\ s.t.\ c(w_{i-1}, w') = 0$
- **Back-off Smoothing**: use lower order n-grams if higher ones are too sparse, $P_{bo}(w_i \mid w_{i-n+1} \cdots w_{i-1}) = d_{w_{i-n+1} \cdots w_i} \frac{C(w_{i-n+1} \cdots w_{i-1} w_i)}{C(w_{i-n+1} \cdots w_{i-1})}$ if $C(w_{i-n+1} \cdots w_i) > k$ else $\alpha_{w_{i-n+1} \cdots w_{i-1}} P_{bo}(w_i \mid w_{i-n+2} \cdots w_{i-1})$, d discounting factor and α back-off factor to ensure probabilities sum to 1
- **Interpolation Smoothing**: combines different n-grams and models, $\sum_j \lambda_j = 1$, $\hat{P}(w_i|w_{i-1}, w_{i-2}) = \lambda_1 P(w_i|w_{i-1}, w_{i-2}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_3 P(w_i)$
- **Fixed-context Language Models**: solve similar words being similar only if they appear in overlapping sequences, input x is mapped to lower dimensional space $h = tanh(Wx + b)$, output layer projects on vocab space $z = Uh$, final prediction w/ softmax $softmax_i(z) = \frac{e^{z_i}}{\sum_k e^{z_k}}$ (so we will never predict 0). Different weights used at different positions even if it's the same word, can't encode long-range dependencies because of the fixed-context (increasing it means increasing the weight matrix)

RNNs

- **RNNs**: NN with feedback loops, learns from the entire sequence history. Each step $h_t = \sigma(W_{hx} x_t + W_{hh} h_{t-1} + b_h)$ where h_{t-1} is the memory, W, b parameters shared across all steps. Final output first projected on vocab space then classifier $z_t = \sigma(W_{zh} h_t + b_z)$, Elman network specification. Struggles with long-range dependencies because memory is overwritten at each step. We can output something at each step (sequence labelling). Recurrent so cannot be parallelized. For backprop go through the same steps you went through when moving forward, $\frac{\partial z_t}{\partial h_{t-2}} = \left(\frac{\partial \sigma(v_t)}{\partial v_t} W_{zh}\right) \left(\frac{\partial \sigma(u_t)}{\partial u_t} W_{hh}\right) \left(\frac{\partial \sigma(u_{t-1})}{\partial u_{t-1}} W_{hh}\right)$
- **Multiple Layers RNNs**: at each step all layers use the output from the previous step of their respective layer and the output of the previous layer in that same step. Final output comes from the top layer of the last step, like in CNNs higher layers have broader understanding while lower ones focus on details and locality
- **RNN bidirectionality**: concatenate or aggregate by mean/max the outputs of all layers at each step. Each direction has own learnable params, access to future text
- **Vanishing Gradients in RNNs**: w/ long inputs drive gradients to 0 $\rightarrow$ learning stops, $\frac{\partial h_t}{\partial h_{i-T}} = \prod_{i=t-T}^{i=t} \frac{\partial h}{\partial h_{i-1}} = \prod_{i=t-T}^{i=t} \frac{\partial \sigma(u_i)}{\partial u_i} W_{hh}$, since W_{hh} is small as used for regularization and the derivative for most activation functions $\frac{\partial \sigma(u_i)}{\partial u_i} < 1$
- **Gated RNNs**: use gates to avoid vanishing gradients $h_t = (h_{t-1} \odot f) + activation(x_t)$, gate value $f \in (0, 1)$ decides how much info from the previous state is kept. h_{t-1} is not passed to the activation function so4 we don't derive by it
- **Long Short Term Memory RNNs**: h_{t-1} short-term memory, $c_t = i_t \times \tilde{c}_t + f_t \times c_{t-1}$, $\tilde{c}_t = \phi(W_{cx} x_t + W_{ch} h_{t-1} + b_c)$ cell state that tracks longer-term dependencies (e.g. semantic behaviour), $f_t = \sigma(W_{fx} x_t + W_{fh} h_{t-1} + b_f)$ forget gate that controls how much memory is forgotten, $i_t = \sigma(W_{ix} x_t + W_{ih} h_{t-1} + b_i)$ input gate that controls how new info is added to the state memory (not only how much because it uses weights and biases for manipulation), $o_t = \sigma(W_{ox} x_t + W_{oh} h_{t-1} + b_o)$ output gate that decides how the hidden state will influence gate values at the next step, final output $h_t = o_t \times \phi(c_t)$. Gradients will not vanish if forget gate is well above 0
- **Gated Recurrent Unit RNNs**: generally $h_t = (1 - z) \odot h_{t-1} + z \odot act(x_t, h_{t-1})$, specifically $act(x_t, h_{t-1}) = \sigma_h(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$ where $z_t = \sigma_g(W_z x_t + U_z h_{t-1} + b_z)$ update gate and $r_t = \sigma_g(W_r x_t + U_r h_{t-1} + b_r)$ reset gate. Less powerful than LSTM but faster to train and sometimes better

seq2seq

- **Encoder-Decoder Model**: encode a sequence w/ encoder, use representation produced to seed unidirectional decoder that decodes/generates a sequence. Autoregressive (generates one word at a time). Decoder can be bidirectional, finding data is hard because it consists in pairs of aligned sequences. Training like LMs, minimize average cross-entropy over all tokens $\mathcal{L} = \sum_{t=1}^T -log(P(y_t|y_1, \dots, y_{t-1}, x_1, \dots, x_n)$, gradients propagated through encoder and decoder
- **Attentive Encoder-Decoder**: solve not learning long-range dependencies due to the state being a single vector manipulated each time. At each decoder step use partial outputs produced at each encoder step to compute an attention/mixture (weighted average over those partial outputs), combine it with the final encoder representation to improve the decoder's outputs. Give idea to decoder on "what to decode". We can focus on different things at each decoder step because the attention distribution α_n changes every time since we attend to the current decoder step partial output
- **Attention**: compute pairwise similarity between each encoder hidden state h_n^e (keys) and decoder hidden state h_n^d (query) $a_i = f(h_i^e, h_1^d)$ (raw attention/pairwise scores), convert it to a probability distribution using softmax over the keys $\alpha_n = \frac{e^{a_n}}{\sum_j e^{a_j}}$, compute weighted average $\tilde{h}_i^d = \sum_{n=1}^N \alpha_n h_n^e$ (h_n^e is the value). The raw attention can be multiplicative ($a = h^e W h^d$), linear ($a = v^T \phi(W[h^e; h^d])$) or scaled dot product ($a = \frac{(W h^e)^T (U h^d)}{\sqrt{d}}$). Helps pair input and output words by meaning just by looking at their attention values. Agnostic to the type of RNN used, range $[0, 1]$

Pretrained Data

- **Filtering pipeline**: consider web domain particularities, inspect top 10 urls per domain, deterministic w/ hard thresholds $\rightarrow$ strong decision points, stochastic smoother
- **Quality heuristics for filtering**: count of chars/words in a sentence/document, repetition count of instances of chars/words, existence of a word/sentence in the word/document, ratio of certain words in a document, statistics (mean length and std of sentences). Controlled and robust but surface level, requires thresholds
- **ML-based quality filtering**: given examples of good/bad docs do classifier-based quality filtering (fastText) or perplexity based filtering, hard to estimate impact
- **Deduplication**: exact (check substrs or sentences) or fuzzy (BLOOM filters or MinHash for speed-memory trade-off), may lead to keeping only bad data (because you remove all documents except one containing a sentence, the one you keep is spammy)
- **Data quality evaluation**: train small 1/2B model on 30B tokens, use high-signal benchmarks like commonsense QA (sensible to data quality, as data/model quality improves the benchmark's scores should monotonically increase with low variance). Smaller models check normalized LL to check correctness of answers, for larger models ask exact letter answers (e.g. A/B/C) and check
- **Synthetic data contamination**: after ChatGPT release the percentage of synthetic text has gone up together with overall quality (e.g. Cosmopedia made by Mistral 8B)

Lab Exercises

- Beam search, self-consistency, quantization, CoT, thinking budget is test-time scaling
- w/ RLVR at start most problems not solved, can get close but if not exact 0 reward
- Min 512 reasoning tokens to see improvement which is logarithmic, plateau at 2-4k
- Thinking budget should be adapted depending on task, deployment and confidence
- $D_{\text{KL}}^{(i,t)} = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,:t})}{\pi_\theta(o_{i,t} \mid q, o_{i,:t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,:t})}{\pi_\theta(o_{i,t} \mid q, o_{i,:t})} - 1$ $\quad \delta_t = \log \pi_{\text{ref}} - \log \pi_\theta$ $\quad D_{\text{KL}}^{(i,t)} = e^{\delta_t} - \delta_t - 1$

Transformers

- **Transformer**: encoders and decoders made of transformer blocks that build compositional representations of input. Each block starts with multi-headed self-attention, its output is combined w/ Residual connection + layer normalization ($LayerNorm(x) = \frac{x - \mathbb{E}[x]}{\sqrt{Var[x] + \epsilon}} * \gamma + \beta$, $LayerNorm(x + Sublayer(x))$), after that a FFN, its output is combined w/ Residual connection + layer normalization. At start of encoder and decoder compute embeddings + positional encoding. At end of decoder do linear projection + softmax to output probabilities for next token
- **Decoder Transformer block**: same as normal but adds at the start zero-masked multi-headed self-attention (during training we don't want to use future tokens $s < t$, set raw attention $a = -\infty$ so final softmaxed attention is 0) w/ Residual connection + layer normalization, after that do cross-attention instead of self
- **Self-Attention**: used in Trasformers (not in RNNs) to make them non-recurrent and parallel, stop computing attention distribution over decoder hidden states, do over encoder hidden states instead. Use as query each time a different encoder hidden state and as keys/values all the others, $a_{st} = \frac{(W^Q h_s^l)^T (W^K h_t^l)}{\sqrt{d}}$, $\alpha_{st} = \frac{e^{a_{st}}}{\sum_j e^{a_{sj}}}$, $\tilde{h}_s^l = \sum_{t=1}^T \alpha_{st} (W^V h_t^l)$. In matrix (all-at-once) form $a = \frac{(W^Q q)(W^K K)^T}{\sqrt{d}}$, $\alpha = softmax(a)$, $\tilde{h}^l = W^O \alpha (V W^V)$, $q = h_s^l$, $K = V = \{h_t^l\}_{t=0}^T$. Attention can be computed for all different queries in parallel
- **Multi-headed Attention**: use many different weight matrices to compute different attention heads/values over the same queries and keys (each could focus on different things), combine results. $a_{h,t} = \frac{(W_h^Q Q_t)(W_h^K K)^T}{\sqrt{d/H}}$, $\alpha_{h,t} = softmax(a_{h,t})$, $M_{h,t} = \alpha_{h,t}(V W_h^V)$, $M_t = W^O [M_{1,t}; \dots; M_{H,t}]$, $W_h^Q, W_h^K \in \mathbb{R}^{d/H \times d}$, $W_h^V \in \mathbb{R}^{d \times d/H}$ instead of $\mathbb{R}^{d \times d}$
- **Cross-Attention**: use output of final encoder block as keys and values, use as query the output of the masked multi-headed attention in the decoder
- **Positional Encoding**: sinusoid function offset by a phase shift proportional to the position in the sequence, in practice learning embeddings is better
- **Positional Embedding**: only tradeoff is poor generalisation to sequences longer than the maximum positional embedding you learned

Tokenization

- **Tokenization**: turning stream of text into pieces (tokens, useful semantic units for processing), structures unstructured text for machine learning. Selectively pick tokens to represent, discarded tokens (e.g. low volume ones) mapped to <UNK>
- **Word Tokenization**: tokens = words, use natural breaks and delimiters. Requires special rules, difficult to implement for languages without spaces (e.g. Chinese), treats different forms of the same word differently (power law distribution, huge embedding matrices). Can't process out-of-vocab words, struggles with slang and typos
- **Character Tokenization**: small vocab, no OOV, lengthy inputs slow down training and inference. Difficult learning relationships between chars that form words
- **Subword Tokenization**: split text into subwords/n-grams so that frequent ones are not split, rare words should be decomposed in meaningful subwords. Token learner makes vocab starting from raw training corpus, token segmenter takes raw test sequences and tokenizes them w/ greedy longest match according to vocab. Meaningful individual tokens (not necessarily corresponding to meaningful indivisible morphological units (morphemes)), not optimal for agglutinative or non-concatenative langs (e.g. Turkish and Arabic), manageable vocab size, different forms of same word treated similarly (e.g. split off "s" at the end of "talks" so that it matches "talk"), reduces OOV (as we should be able to split input text according to our vocab). # of subwords should be proportional to dataset size and related to tasks to be done and their domains. Larger vocabularies slow down training but speed up inference
- **Byte Pair Encoding**: initially data compression algorithm, represent dataset with least tokens. Split text corpus in words (pre-tokenization, append at end of each word </w> to tell if char is at end of word), split words in chars, create base vocab from unique chars. Iterate: compute pair frequency of tokens, merge most frequent pair, add to vocab (don't remove the 2 tokens from the vocab, so that we can represent any text alongside big meaningful tokens). Repeat until desired size/# of iterations
- **WordPiece**: merge pairs based on score = (frequency of pair) / (frequency of first element * frequency of second element)
- **SentencePiece**: train subword models directly from sentences (no pre-tokenization), encode everything as Unicode (also whitespaces)
- **Byte-level Tokenization**: represent text using byte sequences resulting from UTF-8 encoding, very robust to noise, OOV and out-of-distribution data. Base vocab is size 256 (all possible byte values), final output is rendered as Unicode chars starting from the bytes that make up tokens (UTF-8 can render any Unicode char using the tagging schema (first bits every few bytes)). Works even for new, unknown Unicode chars, longer training and inference due to longer sequences. Used in Byte-level BPE (used for first GPTs) and ByT5 (less parameters than mT5 based on SentencePiece)

Lab Exercises

- Beam search amplifies high-prob low-quality tokens, repetitive as it maximizes joint probability over the sequence, only a bit more coherent than greedy. Fix by changing prob distribution to encourage diversity (high temp, top-p/top-k sampling, penalize or don't repeat common n-grams (but can impact legit repetitions, or generate odd words)). Different beams differ at surface-level like word choice, writing aid
- Good human text does not follow the distribution of high prob next words, it's desirable for the generated text to be surprising (use stochastic algorithms like top-p/k)
- Beam search works w/ predictable length tasks like translation, not open-ended tasks
- Random Sampling: no sequence-level planning, incoherent/garbled outputs, no fluency and factual consistency, can select low-probability tokens that break context
- Models can reach correct answer through flawed reasoning (few-shot CoT solves it)
- Zero-shot prompts can perform better than few-shot if examples are biased
- Bradley-Terry answer preference model $p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$
- GRPO assigns same outcome-level reward to all tokens in response, + interpretable

Pretrained Language Models

- **ELMo**: calculate contextualized token embeddings using hidden states obtained during training on large corpus a two-layer bidirectional LSTM-based LM. Pretrain objective: $\sum_{k=1}^{N} \log p(t_k \mid t_1, \ldots, t_{k-1}; \Theta_x, \overrightarrow{\Theta}_{LSTM}, \Theta_s) + \log p(t_k \mid t_{k+1}, \ldots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s)$, Θ_x, Θ_s shared parameters for input embeddings and projection on vocab space + softmax for prediction. 1B Word Benchmark was used to evaluate performance, poor choice because made of randomly shuffled single sentences (long-range dependencies ignored). Weighted average of the hidden state of each internal layer because useful representations may be at different layers and we want to factor them in separately, for a specific task $\text{ELMo}_k^{task} = \gamma^{task} \sum_{j=0}^{L} s_j^{task} \mathbf{h}_{k,j}^{LM}$. No fine tuning, you just learn γ^{task} (scaling factor helpful for task adaptation) and s_j^{task} (softmax-normalized weights, each task attributes different weights to each layer). At lower layers focus on word/sentence embeddings (morphology), at higher levels we have concatenated hidden states from the two unidirectional models (ELMo not bidirectional) that capture complex context and meaning
- **Feature-based Transfer Learning**: during pretraining learn embeddings that can be used to seed a downstream model (like ELMo does by appending an extra (context) representation to the input), during fine-tuning design a new model architecture whose embeddings are initialised with the pretrained ones and train a model on a task
- **Causal Language Modeling**: left to right, autoregressive, better text generation
- **Masked Language Modeling (MLM)**: mask out random tokens, better classifiers
- **BERT**: transformer encoder, vocab 30k, pretraining data = pairs of naturally occurring sequences of text w/ 15% masked tokens (80% w/ [MASK], 10% w/ random token, 10% left the same). Different attention heads attend differently, broadly, to next token, to [SEP], to punctuation. Uses segment (A/B) and position embeddings, improves transformer-based contextual representations using MLM and bidirectional encoding (attention). 13GB training data (RoBERTa "Robustly Optimized"160GB). In finetuning replace pre-training mask prediction head w/ task-specific linear layer (uses [CLS] at start of sequence for classification, final token embeddings for sequence labelling). BERT can produce static embeddings like ELMo but better w/ finetuning
- **Finetuning Transfer Learning**: learn model that can be finetuned like GPT/BERT, uses small labelled meaningful datasets, train it further on a specific downstream task by computing gradient of loss w.r.t the weights and update them. Cheaper, multiple runs, embeddings learned also during finetuning not just pretraining
- **Pretraining**: slow, expensive, simple objectives, tons of data, model not useful yet
- **Whole word masking**: instead of tokens/subwords, helps making the prediction task difficult enough, forcing the model to learn well
- **DistilBERT**: very small knowledge distillation over soft target probabilities of BERT (prob distribution over vocab) + do standard MLM learning, 97% performance, $\mathcal{L}_{distil} = -P_{BERT}(y_t^*|[M], \{y_s^*\}_{s\neq t}) log P_{dbert}(y_t^*|[M], \{y_s^*\}_{s\neq t})$, $\mathcal{L}_{mlm} = -log P_{dbert}(y_t^*|[M], \{y_s^*\}_{s\neq t})$, $\mathcal{L}_{tot} = \gamma_1 \mathcal{L}_{distil} + \gamma_2 \mathcal{L}_{mlm}$
- **ELECTRA**: BERT only learns from masked tokens (15% of data), do GAN instead: Generator is a small MLM like DistilBERT; Discriminator (ELECTRA) predicts whether a token is corrupted, this way we learn from all tokens simultaneously
- **GPT**: generative pretrained transformer (decoder transformer), generates text instead of embeddings like BERT, masked multi-headed self-attention, no cross-attention (decoder-only). 13GB training data segmented in 512 tokens windows. Pretraining task is next word prediction, minimize NLL of sequences in the dataset $\mathcal{L} = -\sum_{t=1}^{T} log P(x_t^*|\{x_s^*\}_{s<t})$. During finetuning minimize cross-entropy of labels assigned to samples $\mathcal{L} = -log P(y^*|X)$. Less expressive representations because unidirectional, first transformer model for contextualized embeddings that generates text
- **GPT2**: same architecture, 40GB data, less finetuning more zero-shot prediction
- **BART**: classic transformer, bidirectional encoder feeds into autoregressive decoder (cross-attention), pretraining combines BERT (MLM-like text corruption before encoder) and GPT (next word prediction objective). Can do classification and sequence labelling as good as RoBERTa by giving same sentence to encoder (w/ [CLS] appended) and decoder (w/ output representation classification). Trained on a lot more data than BERT, tested different type of text corruption for MLM pretraining: masking, deletion, document rotation, sentence order permutation, text infilling (masking but can span for more tokens). Last 2 the best combination
- **T5**: seq2seq, similar to BART with text infilling, philosophy: any task can be done with seq2seq. During pretraining many things were tried, findings used to train 11B model (e.g. MLM approach, corruption strategies/rate and span length)
- BART & T5: seq2seq transformers extendable to all tasks, learn to generate tokens in language of label space, instead of learning to interpret output embeddings of [CLS]

Token Decoding

- **Autoregressive model**: input $\{y_{<t}\}$, output $\hat{y}_t$, f model w/ parameters θ, output scores $S = f(\{y_{<t}\}, \theta)$, output probs $P(y_t = w|\{y_{<t}\}) = \frac{exp(S_w)}{\sum_{w' \in V} exp(S_{w'})}$
- **Maximum Likelihood Training**: teacher forcing, objective $\mathcal{L} = -\sum_{t=1}^{T} log P(y_t^*|\{y_{<t}^*\})$, y_t^* truth/gold token. does not encourage diversity, fix by adding data or learning from sequences generated by the model itself
- **Greedy Decoding**: cannot revise prior decisions, maximum likelihood training, ends in loops repeating very common words that decrease NLL (e.g. "I don't know")
- **Argmax Decoding**: greedy, pick token with highest probability
- **Beam Search**: track b (beam size) most probable sequences at each step, uses b times more computation and memory to keep track of paths
- **Random Sampling**: sample token from the softmax probability distribution
- **Top-k Sampling**: random sampling of top k tokens per probability
- **Top-p (nucleus) Sampling**: random sampling of most probable tokens making up p probability mass, top-k with dynamic k based on how probability is distributed
- **Softmax temperature**: make softmax probability distribution sharper ($\tau < 1$) or flatter ($\tau > 1$, more diverse outputs), $P(y_t = w|\{y_{<t}\}) = \frac{exp(S_w/\tau)}{\sum_{w' \in V} exp(S_{w'}/\tau)}$
- **N-gram statistics rebalancing**: cache database of sentences from training, at decoding search for most similar sentences to current context, rebalance prob distribution using probability distribution of words following sentences similar to the context
- **Re-ranking**: decode many sequences, score (multiple, e.g. ppl & style) and re-rank
- Decoding is important because training perfectly calibrated models it's really hard

Benchmarks & Evaluation

- **Dataset**: manifestation of task (instantiation of problem) w/++ input-output pairs
- **Benchmark**: dataset collection for performance evaluation of models, measure algorithmic innovation. Should use historical real data because synthetic/machine-generated data could have bias. Universal and popular, alternative is cherry picking test data with properties that validate the algorithm. Diverse benchmarks from diverse domains prove generality. May not translate to real-world
- **Natural Language Inferences**: NLI, given premises classify an hypothesis as entailment (true), neutral or contradiction (false), hypothesis should be based on premises
- **Benchmarks Building**: define task (e.g. NLI), design annotation guideline to collect a dataset (explain to annotators the task they will do), run pilot studies to refine guideline (w/ experts, prioritize annotators engagement to boost quality), analyse the initial data after more pilot studies (w/ crowdworkers), collect data at scale (pay well, disqualify high disagreement annotators and samples)
- **Annotation Artifacts**: to easily create neutral sentences annotators add info to hypothesis, to create contradictions they add negations. Models can cheat and perform well by looking for extra info or negations. Cause generalisation issues especially when datasets are drawn from different distributions than real world data. Asking really unlikely things could confuse models that are used to give likely answers
- **Spurious Correlation**: words heavily associated with labels but without causation
- **Shortcuts**: models learn them instead of the task, then in real world they don't work
- **Annotator Bias**: few annotators create many datasets and may use heuristics to annotate data quickly, knowing the annotator for a sample provides a shortcut
- Other Annotation Biases: Undesirable Behaviour, Harms, Inductive/Statistical Bias
- **Manual Dataset Re-balancing**: post-hoc, so certain answers are not predictable only from the question (e.g. have pictures of both men and women wearing glasses)
- **Contrast (Data)Sets**: test specific dimensions of what we want in the first place, perturb examples using known patterns (keep same text but swap negative words)
- **Data Augmentation**: new examples to demonstrate new contexts, diversification!
- **Adversarial Filtering Algorithms**: remove examples with shortcuts/easy so that models don't learn shortcuts. Train classifiers on different splits and evaluate selected validation examples, remove examples very easily solved by one particular classifier
- **Bias Mitigation**: train small bias-only/absorbing model (only has hypothesis, tries shortcuts) with actual NLI model (solves only samples not solvable by biased model)
- **Datasheets for Datasets**: framework for recording data details about datasets
- **Data Cascades**: small mistakes in early stages cascade and compound to downstream
- NLP becoming mainstream showed that many popular benchmarks didn't translate
- **Leaderboard Illusion**: small differences in benchmarks that determine standings are often caused by prompt formatting, dataset contamination (training on benchmarks), etc... depending on how robust the benchmark is
- **Science of Evaluation**: designing reliable experiments that measure model capabilities trustfully. Different evaluations for different needs, determines how future AI is built. Define model usage, outputs evaluation and results interpretation
- **Downstream performance evaluation**: old ways (n-gram precision/recall, synonym matching) fail because semantic equivalence $\neq$ lexical overlap and many correct answers, now (knowledge understanding, reasoning, coding, IF, agentic (e.g. MMLU))
- **Probabilistic (log-prob) evaluation**: measures how likely a model thinks the correct answer is compared to other ones. Compute log-probability of candidate answers for a prompt by summing token log-probs to get sequence likelihood, choose highest likelihood answer (e.g. HELM). Deterministic, comparable, measures performance, works well for structured tasks, requires access to the logits (not just an API)
- **Evaluating Instruction Following**: tough to evaluate open-ended generations (coherence, intent, tone, format requirements, helpfulness), e.g. ChatbotArena
- **LLM-as-a-judge**: replace not scalable and biased human evaluations with strong LLMs, fast, cheaper, somewhat representing. Prompt sensitive and self-preference bias. Compare two responses, assign score, evaluate criterias or consortium
- **Evaluating Agentic Behaviour**: SWEBench (real GH issues), CyBench, MLEBench
- **Evaluating Jailbreaking/Safety**: attack success/refusal rate, average attempts, model-based harmfulness score, tradeoff between helpfulness and refusal
- **Self-Consistency**: improve CoT by sampling many reasoning paths, selecting most freq. answer, accuracy boost by find agreement between diff. reasoning trajectories
- Use different benchmarks at different training steps, evaluate base models on log-prob, perplexity; aligned models (post SFT, RL) on IF, LLM-as-a-judge, etc...
- **Content overlap metrics**: compute similarity score between generated and gold-standard text (e.g. human written), fast and efficient but not good enough alone. Can be n-gram and/or semantic (SPIDEr = SPICE + CIDEr is both), $\in [0, 1]$
- **N-gram overlap metrics**: correlated with human judgement, worse for open-ended tasks (e.g. dialogue, summaries), e.g. BLEU (used for translation, n-gram precision + length comparison w/ brevity penalty, tokenization dependant), CIDEr
- **Semantic overlap metrics**: way better for translation, PYRAMID (human content selection variation when evaluating summaries) SPICE (semantic propositional image caption evaluation, abstract scene graph representation from caption) SPIDEr
- **Model-based metrics**: learned representations to compute semantic similarity between generations and reference texts, good, not interpretable. Vector Similarity (using embeddings), Word Mover's Distance (between sequences, word embedding similarity matching), Sentence Mover's (based on previous, evaluate text in continuous space using sentence embeddings from RNNs), BERTScore (pretrained contextual BERT embeddings, match words by cosine similarity), BLEURT (regression model based on BERT scores grammatical and meaning quality), LLMs (define a rubric)
- **Human Evaluations**: most important, especially if done by researcher, slow, expensive, difficult to conduct (lazy, fraudolent, incosistent people). Automatic metrics must correlate with human evaluations, can't be compared across differently-conducted studies. Humans require 10-20 examples to calibrate ratings, more difficult to rate LLM text, rubrics (structured scoring guides) helped

Lab Exercises

- **Pattern Exploiting Training (PET)**: NLP task reformulated to a cloze style (gap fill) task for few shot learning, tune a linear classification head (mostly MLP layer attached on top of pretrained model) instead of training the whole model on classification task. Define verbalizer tokens (w/ label assigned to each), to classify sequences make prompt template so that next token will represent the sequence (e.g. "[movie review]. Overall, the movie was <mask>"), sample next token from verbalizers tokens only, report label assigned to the verbalizer generated. Make verbalizers single token and close to most natural words humans use. Example of In-context learning

Model Scaling

- **Language Model Scaling**: logarithmic scales for # parameters, performance improves (bigger memory in form of parameters) as seen in TriviaQA dataset
- **Emergence**: quantitative changes in the model result in qualitative changes in behaviour, abilities appearing only in bigger models (e.g. good Machine Translation on low-resource languages with models not trained for it)
- **Test-time scaling**: use more compute during inference, leads to way better performance and reasoning. </think> token removed to keep thinking, added to stop
- Finetuning can teach models to generate narrative output/stories, no special prompts

Prompting

- **Zero-shot prompting**: solve problem given only task description and prompt
- **In-context Learning**: emergent ability, model learns/understands how to do a task based on labelled examples given as input w/ task description and prompt. Very large models benefit more from it. Smaller models sensible to example selection/order. No theory on why it works, just that frequent terms in pretraining are easier to parse. Improve formatting (very important, more than correctness for CoT examples) answers
- **Prompt Engineering**: craft prompts (e.g. cheese =>) that improve performance
- **AutoPrompt**: gradient-guided search for pattern tokens that make best prompt, initialize trigger tokens, estimate change in loss from replacing trigger tokens with other ones. Random words can decrease loss (like jailbreaking where we pick unconventional inputs to get desired output). Trigger tokens merged in input using a template different for each task. Classification on [MASK] token embedding to determine label
- **Prompt Tuning**: improve embeddings/representation of prompts to decrease loss, initialize special prompt vector (live in token embeddings space but are mapped to a prompt instead of a token), prepend them to any task sample, compute gradients w.r.t special prompt vector and update them (rest of model stays frozen). More efficient training (less data than SFT) and separable multi-task learning. Special prompt vector also called soft prompt because not more interpretable (it's random tokens)
- **CoT Prompting**: include reasoning traces in few-shot prompts before the label

Posttraining

- **Exposure Bias**: when training by maximum likelihood we force the model to generate the most probable tokens, knowing exactly which one it is. At inference we don't know it so we can't guide the model and it eventually goes crazy losing coherence
- **REINFORCE**: cast model as Markov decision process, state s representation of current context, actions a words that can be generated, policy π decoder, rewards r external score (can be same as evaluation score). Learn behaviours (e.g. consistency, politeness (count polite words, exploitable)) by rewarding model when it exhibits them, use any behaviour that can be made into a reward function (reflect human quality, not exploitable). Sample sentence, optimize $\mathcal{L}_{RL} = -R(\hat{Y}) \sum_{t=1}^{T} log P(\hat{y}_t|\{y^*\}; \{\hat{y}_{<t}\})$ to increase prob. (how much determined by reward) of sampling the current token w/ this same context, rewards whole sequence. Can reward differently at each step $\mathcal{L}_{RL} = -\sum_{t=1}^{T} (r(\hat{y}_t) - b) log P(\hat{y}_t|\{y^*\}; \{\hat{y}_{<t}\})$, a bias is added for stability through variance reduction (alternatively joint optimisation $\mathcal{L} = \mathcal{L}_{MLE} + \alpha \mathcal{L}_{RL}$, no Catastrophic (pretraining) Forgetting). No exposure bias as it teaches model how to recover from generating a bad token (instead of following it and breaking down)
- **Houlsby Adapters**: efficient finetuning, keep pretrained weights frozen, initialize and update new mid-size FFN between transformer blocks layers, manipulate output
- **LoRA**: keep pretrained weights frozen, initialize new FFN alongside layers in transformer blocks, FFN size $2dr$, r hidden dimension, add on top of output like RNNs
- From GPT3 (pretrained base model) to ChatGPT: SFT with human-annotated demonstration data (13k samples), Reward model (33k samples), RLHF PPO (33k)
- **Supervised Finetuning (SFT)**: optimize $\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t^*|\{y_{<t}^*\}; \{x^*\})$, T answer length, $\{x^*\}$ instruction tokens, y_t^* gold token, $\{y_{<t}^*\}$ tokens already generated. Minimize NLL of generating human-annotated response. Expensive, exposure bias, good at giving plausible answers that lack factuality
- **Reward Model**: predicts quality score for model's outputs (demonstrations), trained on human made rankings of multiple outputs for same input (asking for exact score is too biased). Trained to give higher scores to outputs ranked higher by humans $\mathcal{L}_{RM} = -log\ \sigma(R(Y_+) - R(Y_-))$, as $R(Y_+) - R(Y_-) \to \infty$, $\sigma(\cdot) \to 1$, $\log(\cdot) \to 0$, naive approach is generating a bunch of outputs and reporting the highest-scoring
- **RL from Human Feedback (RLHF)**: train reward model to update model w/ RL
- **Proximal Policy Optimization (PPO)**: keep base/ref model, helps models perform many tasks and feel personalized. Sample answer to a prompt and optimize $\mathcal{L}_{PPO} = -\min\left(R(\hat{Y}) \sum_{t=1}^{T} \log \frac{P_{curr}(\hat{y}_t|\{x^*\};\{\hat{y}\}_{<t})}{P_{base}(\hat{y}_t|\{x^*\};\{\hat{y}\}_{<t})}, g(\epsilon, R(\hat{Y}))\right)$, g clips probs ratio to $[1-\epsilon, 1+\epsilon] \to \mathcal{L} = 0$ so that we don't drift off too quickly from base model + stable loss. Value/critic model estimates rewards for not fully generated answers (advantage) $\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$, $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- **Direct Preference Optimization (DPO)**: no reward model (less flexible than PPO, works in practice), high β diverge less, optimize directly for human preferences $\mathcal{L}_{DPO} = -\log\sigma\left(\beta \log \frac{P_{curr}(Y_+|x^*)}{P_{base}(Y_+|x^*)} - \beta \log \frac{P_{curr}(Y_-|x^*)}{P_{base}(Y_-|x^*)}\right)$, encourages ratio between P_{curr} and P_{base} to be higher for the sample Y_+ than the sample Y_-
- **RL w/ Verifiable Rewards (RLVR)**: no reward model (main issue w/ RLHF, expensive, noisy, exploitable), verify correctness programmatically as reward
- **Group Relative Policy Optimization (GRPO)**: cheaper than PPO (doesn't use critic/value model as big as policy). Pick a prompt X, policy model generates many outputs $\hat{Y}_1, \ldots, \hat{Y}_G$ scored independently, final score normalized w/ Group Computation ($\tilde{R}_i = \frac{R(\hat{Y}_i) - \text{mean}(\{R(\hat{Y}_j)\}_{j=1}^{G})}{\text{std}(\{R(\hat{Y}_j)\}_{j=1}^{G}) + \epsilon}$ aka advantage). Less noisy and exploitable rewards (if rewards are all equal nothing changes, exploit would need to selectively mess up rewards). All wrong or all right outputs don't change anything. Optimize $\mathcal{L}_{GRPO} = -\frac{1}{G}\sum_{i=1}^{G} \frac{1}{T_i} \sum_{t=1}^{T_i} \{\min\left(\frac{P_{curr}(\hat{y}_t^{(i)}|\{x^*\};\{\hat{y}\}_{<t}^{(i)})}{P_{base}(\hat{y}_t^{(i)}|\{x^*\};\{\hat{y}\}_{<t}^{(i)})} \tilde{R}_i, g(\epsilon, \tilde{R}_i)\right) - \beta D_{KL}[P_{curr}||P_{ref}]\}$, KL Divergence Penalty to not drift away too far from base model. Teach models to use natural language reasoning effectively (test-time scaling)